Towards Strong Privacy Protection for Association Rule Mining and Query in the Cloud

Lin Liu, Jinshu Su, Ximeng Liu, *Member, IEEE*, Rongmao Chen, Xinyi Huang, Guang Kou, and Shaojing Fu

***Abstract*—Efficiently mining frequent itemsets and association rules on the encrypted outsourced data remains a great challenge for the time-consuming ciphertext computations. Nowadays, it has been not well addressed for privacy-preserving frequent itemsets and association rule mining schemes with mining efficiency, dataset, and query confidentiality simultaneously. In this paper, we investigate the study of privacy issues on frequent itemset mining and association rule mining on outsourced data in a two-cloud model, where the data are encrypted and outsourced by multiple owners holding different public keys. We develop several secure computation protocols based on additively homomorphic cryptosystem and additive secret sharing, which enable the clouds could securely mine the frequent itemsets and association rules. Furthermore, we also design two kinds of frequent itemset and association rule query service, i.e., service customers query the cloud-mined results, and service customers query with their own decided threshold. The proposed scheme not only supports the mining process on the data encrypted by multiple public keys without compromising the security of the datasets, query data and query results, but also offline users. In addition, the experimental results show that our query scheme is much more efficient than the state-of-the-art work.**

***Index Terms*—Secure outsourcing computation, association rule mining, frequent itemset mining, cloud data security and privacy, two-cloud model.**

I. Introduction

RECENTLY with the fast development of mobile communication technology, increasingly advanced Internet-of-Things (IoT) devices generate a huge amount of data every day. Such large dataset usually contains a huge value if they can be well analyzed. To deal with the generated data, more and more individuals and organizations have shown a growing interest in the paradigm of data mining-as-a-service [1], [2], [3], [4], [5] in a cloud computing environment. In such a paradigm, an individual or organization, lacking computing resources, storage spaces, or expertise, can easily use a cloud platform to outsource data as well as data mining tasks. Although mining on cloud data could be of huge benefits, the data privacy issue has been a general concern from the user's viewpoint. First, as the data owners, when uploading data to the cloud service provider, they would lose physical control of their data which becomes accessible to the cloud server. However, many of the data the cloud collected are sensitive, such as medical records and financial data. Outsourcing such data directly to the cloud may greatly violate the privacy of individuals and organizations. Second, as the mining service customer, one might want to make his queries and mining results secret to the cloud server, since the query and the mining result may also contain sensitive information. In such a scenario, the security and privacy of the raw data, query data and the mining result should be well considered.

To address such problems, researchers have proposed two kinds of technology, i.e., randomization-based schemes [5], [6] and cryptography-based schemes [1], [4]. The randomization-based schemes usually have high efficiency but without the guarantee of mining results' accuracy, while cryptography-based schemes can apply stronger security levels and accurate mining results. One direct way based on cryptography is to encrypt the data with a fully homomorphic cryptosystem (FHE) [7] before outsourcing. However, FHE usually takes too many computation costs, which is not practical for large-scale dataset mining. Moreover, we also should note that there are a great number of data owners and mining service users involved. A naive solution is to give them the same public key, and they can use such a key to encrypt the data before outsourcing. However, if one of the data owners leaks the private key, all the data are vulnerable to the attacker. Therefore, it is necessary for different data owners to hold different public/private key pairs. Unfortunately, achieving secure and efficient mining on encrypted data by different public keys still has not been well resolved to date.

In this work, we are mainly interested in frequent itemset mining and association rule mining, which are two popular data mining technologies that have been used in market prediction [8], intrusion detection [9], network traffic management [10] and web analysis [11]. The privacy problem in frequent itemset

Manuscript received 17 July 2022; revised 22 February 2023; accepted 18 April 2023. Date of publication 28 April 2023; date of current version 6 September 2023. This work was supported in part by the National Nature Science Foundation of China under Grants 62102430, 62072466, 62102429, 62122092, 62006241 and 62032005, in part by the Natural Science Foundation of Hunan Province under Grant 2021JJ40688, and in part by the NUDT under Grant ZK22-50. Recommended for acceptance by M. Anisetti. *(Corresponding authors: Jinshu Su; Rongmao Chen.)*

Lin Liu, Jinshu Su, Rongmao Chen, and Shaojing Fu are with the College of Computer, National University of Defense Technology, Changsha, Hunan 410073, China (e-mail: liulin16@nudt.edu.cn; sjs@nudt.edu.cn; chromao@nudt.edu.cn; shaojing1984@163.com).

Ximeng Liu is with the College of Mathematics and Computer Science, Fuzhou University, Fuzhou, Fujian 350108, China (e-mail: snbnix@gmail.com).

Xinyi Huang is with the Thrust of Artificial Intelligence, Information Hub, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong Province 510230, China (e-mail: xinyi@ust.hk).

Guang Kou is with the Artificial Intelligence Research Center, National Innovation Institute of Defense Technology, Beijing 100072, China (e-mail: kg5188@163.com).

This article has supplementary downloadable material available at https://doi.org/10.1109/TCC.2023.3269510, provided by the authors.

Digital Object Identifier 10.1109/TCC.2023.3269510

2168-7161 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

mining and association rule mining was first considered by Kantarcioglu et al. [12] and Vaidya et al. [13] in the case of horizontally and vertically partitioned databases respectively. Since then, many works [14], [15], [16], [17] have been proposed in this area. Unfortunately, as far as we know, all the proposed schemes cannot be used for mining data encrypted by different public keys. Recently, there are also some works designed for privacy-preserving frequent itemset and association rule queries. Qiu et al. [1] proposed a frequent itemset query scheme, which supports queries with privacy protection of three different levels. Unfortunately, we found that even the highest privacy level method fails to protect the privacy of the mining result. Moreover, Qiu et al.'s scheme cannot support multi-user settings where the data may be encrypted with various keys. To address these problems, Liu et al. proposed a novel scheme that can protect the mining result well and also support the mining of the data from multiple users [18]. The main downside of Liu et al.'s scheme is the significant computation and communication costs on the cloud, which is mainly due to the huge number of dummy transactions involved in each query. Another limitation of their approach is that their scheme does not support the functionality of mining on outsourced data. Most recently, Pang et al. [19] improved the performance of Liu et al.'s work [18] proposed a secure frequent itemset and association rule query system. However, in their work, it takes about 2164.9 min for one query, which is extraordinarily time-consuming.

A. Motivation

The motivation of this paper can be summarized as "proposing an *efficient* privacy-preserving frequent itemset and association rule query with *a high-security level*." We improved efficiency-security trade-offs in two ways: (1) we were able to get rid of the dummy transactions, which have been commonly used in other related works, while still supporting multi-user and multi-keys; (2) we were able to design a mining cloud to extract important frequent itemset and association rules such that the user can query on the cloud-mined results, instead of querying on the whole dataset stored in the cloud.

Getting Rid of Dummy Transactions: Previous works about privacy-preserving frequent itemset and association rule queries can be divided into dummy transactions needed ones and dummy transactions needless ones. The former usually cannot support multi-user with multi-key, meaning that with a low-security level, since one of the data owner's key leakage will damage the other's data security. In the latter ones, dummy transactions are used to cover the original support of the query transactions. Obviously, adding dummy transactions may arouse unnecessary computations. In this work, we want to design a new algorithm that can protect the support of the query without adding dummy transactions.

Designing Mining Cloud: In most of the previous works, when receiving a query, the cloud will mine the whole dataset for this query to figure out whether it is a frequent itemset or not. To further promote the query process, we try to design a mining cloud, which can mine all the frequent itemsets and association rules on the encrypted dataset. Then, when receiving an encrypted query, the cloud just needs to find out whether it is one of the mined results and then sends the encrypted message to the user, without querying the whole dataset again. In this scenario, the cloud mining process may be time-consuming, but it just needs to be done once and offline.

B. Our Contributions

Our contributions of this work can be divided into three parts, namely:

- We manage to design a scheme that supports privacy-preserving itemset and association rule mining on data encrypted with different keys. That is, our scheme is well suitable for the practical case where the data outsourced to the cloud is from multiple owners with different encryption keys. Moreover, our scheme also enables the cloud to provide privacy-preserving query services at small costs.
- As the building blocks, we design a set of two-party communication algorithms based on additive secret sharing and additively homomorphic cryptosystem, which can efficiently fulfill some basic computations needed during the mining. We then rigorously show that the designed scheme can indeed achieve a higher privacy level than the most recent works on privacy-preserving frequent itemset query systems [1], [4]. A full security analysis of our scheme in the semi-honest model is also provided.
- Finally, we conduct the experiment on different data sets to demonstrate the efficiency of our scheme. We remark that differences in recently published works, i.e., Liu et al.'s scheme [18], Pang et al.'s scheme [19] and Chen et al.'s scheme [20], in this work, no dummy transaction is needed during the mining and query. Consequently, our newly designed scheme significantly improves the efficiency of the query services performed by the cloud.

C. Organization

The remainder of this paper is organized as follows. In Section II, we present the necessary background material on cryptographic tools and frequent itemset and association rule mining. In Section III, we introduce the system model and design goals of our system. Our privacy-preserving solutions for frequent itemsets and association rule mining and query on multiple data owners' outsourced data are shown in Section V, followed by the security and performance analyses in Sections VI and VII respectively. Next, we give the related work in Section VIII. Finally, we conclude our paper in Section IX.

II. Preliminaries

In this section, we introduce essential preliminary concepts, such as frequent itemset and association rule mining, "lite" Cramer-Shoup variant (LCS) cryptosystem[1] [21] [22] and additive secret sharing technology. Table I lists the key notations used throughout this paper.

[1]Note that the LCS cryptosystem was proposed in reference [21] and also described in Section II-D of reference [22].

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATION USED

Notations	Definition
pk_{DO_i}/sk_{DO_i}	Public/Private key of data owner i
pk_M/sk_M	Public/Private key of miner
pk_Σ	The product of all the entities' public key
$[\![x]\!]_{pk}$	Encrypted data x under pk
$\mathbf{MK}$	Master key of LCS cryptosystem
$\mathbf{mDec}_{(pk,\mathbf{MK})}(X)$	Decrypt X with the master key
$\|x\|$	Bit length of x
$supp(X)$	Support of X
$conf(X)$	Confidence of X
L2S	LCS Cryptosystem to Secret Sharing
S2L	Secret Sharing to LCS Cryptosystem
SIP	Secure Inner Product
SCD	Secure Comparison Across Domain
SC	Secure Comparison
ESCD	Extended Secure Comparison Across Domain
ESC	Extended Secure Comparison
SF1I	Secure Find Frequent 1 Itemset Mining
SSC	Secure Support Counting
SFM	Secure Frequent Itemset Mining
SAM	Secure Association Rule Mining
SFQ	Secure Frequent Itemset Query
SAQ	Secure Association Rule Query

A. Frequent Itemset Mining and Association Rule Mining

Frequent itemset mining is the basis of association rule mining which is first proposed by Agrawal et al. [23]. Frequent itemsets are those itemsets whose frequencies in a transaction database are larger than a given minimal support threshold. Here, let I be a set of items, and a set $X = \{x_1, x_2, \ldots, x_k\} \subseteq I$ is called an itemset, or k-itemset with k items. A transaction over I is $t = \{y_1, y_2, \ldots, y_n\}$, where $y_i \in I$. A transaction database is denoted as $T = \{t_1, \ldots, t_m\}$, where m is the total number of transactions. Generally, the relationship between two itemsets is measured by *support* and *confidence*. The support of itemset X, denoted as $supp(X)$, is the possibility of transactions containing X in T. An association rule is of the form $X \Rightarrow Y$ where $X, Y \subset I$ and $X \cap Y = \emptyset$. The *confidence* of rule $X \Rightarrow Y$ is denoted by $conf(X \Rightarrow Y) = supp(X \Rightarrow Y)/supp(X)$. We show the definition of frequent itemset and strong association rules as follows.

Definition 1 (Frequent Itemset): If $X \subseteq t_i$, X is contained by a transaction t_i. $supp_{\min}$ is the user-defined minimum threshold. If $supp(X) \geq supp_{\min}$, X is a frequent itemset.

Definition 2 (Strong Association Rule): A rule $X \Rightarrow Y$ is strong if and only if $supp(X \Rightarrow Y)$ and $conf(X \Rightarrow Y)$ are larger than the given thresholds, i.e., $supp(X \Rightarrow Y) \geq supp_{\min}$ and $conf(X \Rightarrow Y) \geq conf_{\min}$.

B. Apriori Algorithm

Apriori algorithm [24], proposed by Agrawal and Srikant, is one of the most influential frequent itemset mining algorithms. This algorithm is designed by observing the *downward closure* property among frequent $k-$itemsets, i.e., a $k-$itemset is frequent only if none of its sub-itemsets is infrequent. According to this property, we can first mine the frequent k itemsets and then generate the candidate frequent $k+1$ itemsets using it. Moreover, we check all the candidate frequent $k+1$ itemsets and delete these itemsets which didn't meet *downward closure* property. This process iterates until all the frequent itemsets are found in this transaction dataset. Details of Apriori are presented in Algorithm 1. The specific details of the procedures of *apriori_gen* in Algorithm 1 are omitted, which are shown in [24].

Algorithm 1: Apriori Algorithm

Input: D, Dataset. $supp_{\min}$, minimum support.
Output: L, frequent itemset set of D.

```
L_1 = find_frequent_1_itemsets(D).
for all k = 2, L_{k-1} ≠ ∅; k++ do
    C_k = apriori_gen(L_{k-1});
    for all t ∈ D do
        C_t = subset(C_k, t);
        for all c ∈ C_t do
            c.count++;
        end for
        L_k = {c ∈ C_k | c.count ≥ supp_min};
    end for
end for
return L = ∪_k L_k.
```

C. The "lite" Cramer-Shoup Variant Cryptosystem

The "lite" Cramer-Shoup variant cryptosystem (LCS) [21] [22] is an additively homomorphic cryptosystem, with double decryption mechanism, which means that this cryptosystem can provide two independent decryption mechanisms. Under the same public parameter, if given the master key, i.e., $\mathbf{MK}$, of this cryptosystem, all ciphertext encrypted by different public keys can be successfully decrypted. In this article, for simplicity, we use $[\![m]\!]_{pk}$ to represent the ciphertext encrypted by pk. More proofs of the correctness and semantic security of the LCS cryptosystem can be found in [21]. The plaintext domain of the LCS cryptosystem is $\mathbb{Z}_N$, while the ciphertext space is $\mathbb{Z}_{N^2}$. Moreover, each ciphertext of LCS is composed of two integers, i.e., A, B, where $A, B \in \mathbb{Z}_{N^2}$.

As an additively homomorphic cryptosystem, the LCS scheme has the following properties: 1), $\mathbf{Dec}_{sk}([\![m_1]\!]_{pk} \cdot [\![m_2]\!]_{pk}) = m_1 + m_2$. Suppose that $[\![m_1]\!]_{pk}$ (resp. $[\![m_2]\!]_{pk}$) is composed of A_1, B_1 (resp. A_2, B_2). Therefore, $[\![m_1]\!]_{pk} \cdot [\![m_2]\!]_{pk}$ means $A_1 \cdot A_2$ and $B_1 \cdot B_2$. 2) When given any $m, k \in \mathbb{Z}_N$, it can be calculated as $([\![m]\!]_{pk})^k = [\![k \cdot m]\!]_{pk}$. Besides, when $k = N-1$, it can be confirmed that $([\![m]\!]_{pk})^{N-1} = [\![-m]\!]_{pk}$.

D. Additive Secret Sharing

The additive secret sharing (SS) is a kind of secure multiplication computation method introduced by Shamir [25]. The basic idea of SS is to split an integer x into two additive numbers, i.e., $\langle x \rangle = \langle x \rangle^A + \langle x \rangle^B \bmod N'$, where $x, \langle x \rangle^A$ and $\langle x \rangle^B$ are all within the ring $\mathbb{Z}_{N'}$. Moreover, $\langle x \rangle^A$ and $\langle x \rangle^B$ belong to party A, B respectively. To recover such an integer, one of the party should send his share to the other. Receiving the data, an Rec method is needed to recover the original integer, i.e., $x \leftarrow \mathtt{Rec}(\langle x \rangle^A, \langle x \rangle^B)$, where $\mathtt{Rec}(\langle x \rangle^A, \langle x \rangle^B) = \langle x \rangle^A + \langle x \rangle^B \bmod N'$. Note that for simplicity, in the following

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

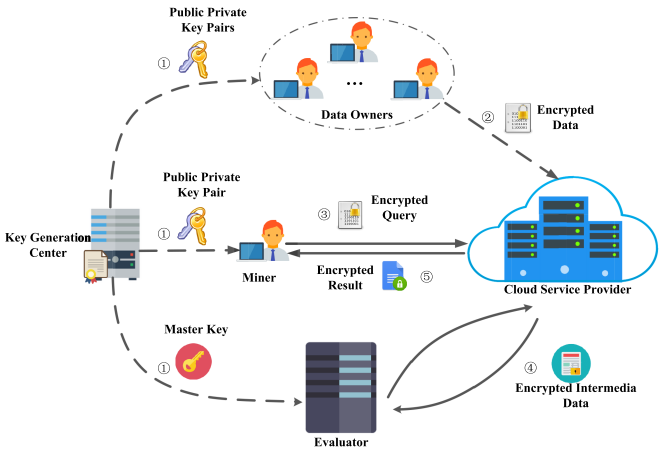


Fig. 1. System model [18].

we omit the modular N' since all the computations are calculated within the ring $\mathbb{Z}_{N'}$.

1) Addition on Additive Shares: There are two kinds of addition methods for the additive shares. The first is the calculation on a shared integer $\langle x \rangle$ and a constant integer $c \in \mathbb{Z}_{N'}$. One of the parties computes $\langle z \rangle^A \leftarrow \langle x \rangle^A + c$, and the other computes $\langle z \rangle^B \leftarrow \langle x \rangle^B$. Thus, the two parties can get $\langle z \rangle \leftarrow \langle x \rangle + c$. The other kind of addition is the sum of two shared integers, i.e., $\langle x \rangle$, $\langle y \rangle$. Party A and B just need to locally calculate $\langle z \rangle^A \leftarrow \langle x \rangle^A + \langle y \rangle^A$, $\langle z \rangle^B \leftarrow \langle x \rangle^B + \langle y \rangle^B$.

2) Multiplication on Additive Shares: The multiplication triples of this scheme is introduced by Beaver [26]. Through running a `Mul`, the two parties get $\langle z \rangle$, where $z = x \cdot y$, and x, y are shared by them. To calculate such a multiplication, a pre-computed arithmetic multiplication Beaver triples is needed, i.e., $\langle c \rangle = \langle a \rangle \cdot \langle b \rangle$. In `Mul`, party A and B locally calculate $\langle e \rangle^A \leftarrow \langle x \rangle^A - \langle a \rangle^A$, $\langle f \rangle^A \leftarrow \langle y \rangle^A - \langle b \rangle^A$, $\langle e \rangle^B \leftarrow \langle x \rangle^B - \langle a \rangle^B$ and $\langle f \rangle^B \leftarrow \langle y \rangle^B - \langle b \rangle^B$. Then they communicate with each other and recover e, f by running `Rec`. Finally, party A (resp. B) calculates $\langle z \rangle^A \leftarrow f \cdot \langle a \rangle^A + e \cdot \langle b \rangle^A + \langle c \rangle^A$ (resp. $\langle z \rangle \leftarrow e \cdot f + f \cdot \langle a \rangle^B + e \cdot \langle b \rangle^B + \langle c \rangle^B$).

In `Mul`, pre-computed Beaver triples are needed. These triples can be generated in two ways, i.e., they can be generated by the two parties through running Oblivious Transfer [27] or a trusted third party [28].

III. System Model and Threat Model

In this section, we give the system model, threat model, and privacy objectives.

A. System Model and Threat Model

In this work, we adpot a widely-used two-cloud model. Same as the system model in reference [18], there are five entities involved in this system, namely Cloud Service Provider (CSP), Evaluator, Key Generation Center (KGC), Data Owner (DO), and Miner. The whole model of this system is shown in Fig. 1.

In this model, we assume that the CSP has collected a great number of data outsourced from the DOs. Then, CSP cooperates with Evaluator to securely mine all the frequent itemsets and association rules. Finally, they together provide a privacy-preserving frequent itemsets and strong association rules query service for the Miner.

Besides, our scheme works in the semi-honest model, where all the parties strictly follow the procedures of the designed cryptographic protocols, but also try to learn others' information. Furthermore, we stress that the CSP cannot collude with Evaluator. This assumption is reasonable since, in a business environment, both of them value their reputations. Note that, CSP could be a governmental organization protecting users' privacy [29], [30]. Such a two-cloud model has been used in many recent privacy-preserving data mining schemes [18], [31], [32], [33], [34], [35]. In this threat model, we also assume that there is an outside adversary $\mathcal{A}$, who trying to obtain the plaintext of the outsourced data and mining results. The adversary $\mathcal{A}$ may eavesdrop on the communication channels and even comprise the DOs, Miners, and one of the clouds. But, we also assume that $\mathcal{A}$ is not allowed to comprise both CSP and Evaluator at the same time. Other details of the system model and threat model are illustrated in reference [18].

B. Privacy Objectives

Besides mining frequent itemset and association rules from the cloud efficiently and correctly, another main purpose of our scheme is to preserve the privacy of all the entities involved.

For our frequent itemsets mining and association rule mining system, CSP should learn nothing about the data other than the final mined frequent itemsets and association rules. The leakage function of this process is denoted by $\mathcal{L}(\kappa, D) = \{frequent_1_itemsets, \ldots, frequent_n_itemsets, A_1 \Rightarrow B_1, \ldots, A_m \Rightarrow B_m\}$, where κ is the security parameter and D denotes the input dataset which are encrypted, $A_i \Rightarrow B_i$ is the mined strong association rule.

For our query system, there are three different privacy levels defined in Qiu et al.'s work [1] based on a *leakage function* $\mathcal{L}$. Here, we present Privacy Level IV, which is achieved in our scheme.

Definition 3 (Privacy Level IV): The Miner submits an encrypted mining request to CSP. All the data uploaded by DOs are encrypted in CSP. During the mining process, none of the CSP or Evaluator knows any information about the mining request and uploaded data. At the end of the protocol, the mining result is encrypted by Miner's public key. Thus the leakage function $\mathcal{L}$ of this scheme can be represented as $\mathcal{L}(\kappa, \vec{x}) = \{\emptyset\}$, where κ is the security parameter and $\vec{x}$ denotes the input data.

IV. System Overview

In this section, we present the basic idea of our privacy-preserving frequent itemset and association rule mining and query system. There are η DOs involved in this mining system. For each DO, the KGC generates the key pairs (pk_{DO_i}, sk_{DO_i}), $i = 1, 2, \ldots, \eta$, and also it generates one key pair, i.e., (pk_M, sk_M), for the Miner. After the key generation, the KGC distributes them to each DO and Miner respectively. Besides, KGC sends the master key of the LCS cryptosystem

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

to the Evaluator. Note that, in this scheme, the public key of each one is revealed to the others. Receiving the key pair, every DO encrypts his data and then uploads the ciphertext to CSP. The work of the DO is done, and no more computation or communication is required for each DO. On one hand, the two-party polynomial computations are efficient when using secret sharing technology, in our system, we may translate the LCS ciphertext into secret shared values if needed. On the other hand, when using secure comparison designed by secret sharing, the communication rounds of the involved parties are huge, meaning that we may use secure comparison designed by LCS rather than secret sharing. Therefore, there is also an N' used for the modular computation during the secret sharing. CSP and Evaluator can generate such an N' through communicating with each other. Note that the plaintext space of LCS is $\mathbb{Z}_N$. Therefore, for the correctness of the computation, we stress that $N' \leq N$.

After receiving the uploaded data, the CSP cooperates with the Evaluator to first mine the frequent itemsets and then the association rules. Specifically, for the secure frequent itemset mining (SFM) scheme, we adopt the Aprior algorithm as the basic mining scheme. Therefore, the basic workflow of our SFM is the same as the Aprior algorithm shown in Algorithm 1. With the mined frequent itemsets, CSP and Evaluator cooperated with each other to mine the strong association rules in a privacy-preserving way. With the mined frequent itemsets and association rules, the CSP and Evaluator can offer two kinds of frequent itemset and association rule query service, i.e., the user-defined threshold and the cloud-defined threshold. In the user-defined threshold case, the user sends the encrypted query and threshold to CSP, which is the same as most of the related works [18], [19], [36]. In another case, the user just sends the encrypted query. In both cases, two clouds cooperate with each other to run secure protocols and return the encrypted query results. Only the user can decrypt the query result.

V. Privacy-Preserving Frequent Itemset and Association Rule Mining and Query

In this section, we describe the construction of our privacy-preserving frequent itemset and association rule mining and query system. First, we present the design details of secure building blocks. Then, the complete outsourcing privacy-preserving frequent itemset and association rule mining and query algorithms are illustrated.

A. Privacy-Preserving Building Blocks

A series of cryptographic building blocks are presented in this section, which are the bases of our scheme. In our context, $pk_\Sigma = pk_M \cdot \prod_{i=1}^{m} pk_{DO_i}$ is used to represent the product of public keys.

1) Secret Sharing to LCS Cryptosystem Algorithm: Suppose that an integer $\langle x \rangle$ is shared by CSP and Evaluator. With this Secret Sharing to LCS Cryptosystem (S2L) method, they can convert it to $[\![x]\!]$. The details are as follows: The CSP and Evaluator encrypt $\langle x \rangle^A$, $\langle x \rangle^B$ respectively. Evaluator sends $[\![\langle x \rangle^B]\!]$ to CSP. Then, CSP calculates $[\![x]\!] \leftarrow [\![\langle x \rangle^A]\!] \cdot [\![\langle x \rangle^B]\!]$.

2) LCS Cryptosystem to Secret Sharing Algorithm: Converting LCS Cryptosystem to Secret Sharing (L2S) is also an essential part of our scheme. CSP picks a random integer, i.e., α, from $\mathbb{Z}'_N$, and sets $\langle x \rangle^A \leftarrow \alpha$. Then, CSP calculates $[\![x - \alpha]\!] \leftarrow [\![x]\!] \cdot [\![-\alpha]\!]$, where $-\alpha = N' - \alpha$, before sending it to Evaluator. Receiving it, Evaluator decrypts it with the master key of the LCS cryptosystem and sets $\langle x \rangle^B \leftarrow x - \alpha$.

3) Secure Inner Product Algorithm: In this subsection, we propose a Secure Inner Product (SIP) algorithm. Suppose that CSP and Evaluator have two additive shared vectors $\langle \vec{X} \rangle$ and $\langle \vec{Y} \rangle$, where $\langle \vec{X} \rangle = \{\langle x_1 \rangle, \langle x_2 \rangle, \ldots, \langle x_n \rangle\}$, and $\langle \vec{Y} \rangle = \{\langle y_1 \rangle, \langle y_2 \rangle, \ldots, \langle y_n \rangle\}$. With this SIP, the two clouds can get additive shares of the inner product of the two vectors. For each $\langle x_i \rangle$ and $\langle y_i \rangle$, CSP and Evaluator run a Mul on them and thus get $\langle x_i \cdot y_i \rangle$. Finally, CSP and Evaluator locally add all the $\langle x_i \cdot y_i \rangle^A$, $\langle x_i \cdot y_i \rangle^B$ respectively and set them as $\langle z \rangle^A$ and $\langle z \rangle^B$, where $z = \sum_{i=1}^{n} x_i \cdot y_i$.

4) Secure Comparison Across Domains Algorithm: In this subsection, we propose a Secure Comparison Across Domains (SCD) protocol, which can be used to securely compare two encrypted integers, i.e., $[\![x]\!]_{pk_M}$ and $[\![y]\!]_{pk_\Sigma}$. Note that, in this algorithm, value range of x, y is $x, y \leq 2^l, l < \|N\|/2 - 1$. Our SCD is based on LCS homomorphic cryptosystem, while Liu et al.'s proposed a SCAD to solve the same problem, which is based on BCP cryptosystem [22]. As we know, the BCP and LCS are both additively homomorphic and support two kinds of decryption. Therefore, this SCD is similar with SCAD. In the following, we just present the basic idea. More details are listed in reference [18].

CSP randomly picks $a \xleftarrow{R} \{0, 1\}$. If $a = 0$, he calculates $C = [\![(2x + 1)]\!]_{pk_M}$ and $D = [\![-2y]\!]_{pk_\Sigma}$. Otherwise, $C = [\![-(2x + 1)]\!]_{pk_M}$ and $D = [\![2y]\!]_{pk_\Sigma}$. Then, he blinds C, D with random number r_a, r_b as $C' = C \cdot [\![r_a]\!]_{pk_M}$ and $D' = D \cdot [\![r_b]\!]_{pk_\Sigma}$ before sending them to Evaluator. Receiving C' and D', Evaluator decrypts them through $\mathbf{MK}$. After that, he calculates $E \leftarrow [\![c' + d']\!]_{pk_\Sigma}$, and sends the encryption result to CSP. CSP calculates $([\![r_a + r_b]\!]_{pk_\Sigma})^{N-1}$, and multiplies it with E. CSP blinds it with two random values r_1 and r_2, which are not larger than 2^l, and $r_2 \ll r_1$, before sending it to Evaluator. The evaluator decrypts it with the $\mathbf{MK}$. If the decrypted value is smaller than $N/2$, he sends $\delta = [\![1]\!]_{pk_\Sigma}$ else he sends $\delta = [\![0]\!]_{pk_\Sigma}$. If the $a = 0$, $[\![t]\!]_{pk_\Sigma} = [\![\delta]\!]_{pk_\Sigma}$ is the comparison result. Otherwise, CSP calculates $[\![t]\!]_{pk_\Sigma} = [\![1 - \delta]\!]_{pk_\Sigma}$ as the final result. If $t = 1$, it means $x \geq y$. Otherwise, it shows $x < y$.

Extension: In some cases, the CSP needs the comparison result to be used in next steps. Our SCD could be easily extended to Extended Secure Comparison Across Domain (ESCD) algorithm. The Evaluator can send the CSP δ directly rather than $[\![\delta]\!]_{pk_\Sigma}$ in step 4.(2), then the CSP gets the comparison result.

5) Secure Comparison Algorithm: Different from SCD, we also propose a Secure Comparison (SC) algorithm which can used to compare two integers encrypted by the same public key. In this SC, assume that CSP owns two ciphertext, i.e., $[\![x]\!]_{pk_\Sigma}$ and $[\![y]\!]_{pk_\Sigma}$, where $x, y \leq 2^l$. After running this SC, CSP can get the ciphertext of the comparison result.

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

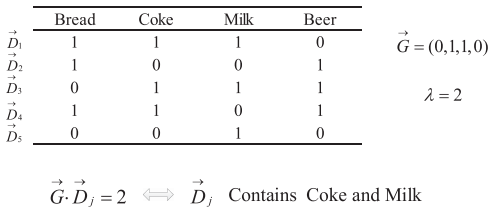


Fig. 2. Secure support counting example.

First, CSP randomly picks $a \xleftarrow{R} \{0,1\}$. If $a = 0$, he calculates $C = [\![(2x+1) - 2y]\!]_{pk_\Sigma}$, else $C = [\![2y - (2x+1)]\!]_{pk_\Sigma}$. Then, he blinds C with the random number r, which is not larger than 2^t, where $2^{(t+l)} < N/2$, before sending it to Evaluator. The Evaluator decrypts it with the $\mathbf{MK}$. If the decrypted value is smaller than $N/2$, he sends $[\![\delta]\!] = [\![1]\!]_{pk_\Sigma}$ else he sends $[\![\delta]\!] = [\![0]\!]_{pk_\Sigma}$. If the $a = 0$, CSP sets $[\![t]\!]_{pk_\Sigma} = [\![\delta]\!]_{pk_\Sigma}$ as the comparison result. Otherwise, CSP calculates $[\![t]\!]_{pk_\Sigma} = [\![1-\delta]\!]_{pk_\Sigma}$ as the final result. If $t = 1$, it means $x \geq y$. Otherwise, it shows $x < y$. More details of the SC can be found in reference [18].

Following the same idea of ESCD, we can propose our Extended Secure Comparison (ESC). For simplicity, we omit the specific details of this ESC algorithm.

B. Secure Frequent Itemset Mining

In this section, we introduce a Secure Frequent Itemset Mining (SFM) algorithm, which can run an Apriori algorithm on the encrypted transaction dataset. CSP and Evaluator cooperate to run this secure frequent itemset mining algorithm. After running this SFM, CSP obtains mining results in plaintext but learns nothing about the original data.

Suppose that Evaluator has the minimum support $supp_{\min}$, and CSP has collected an encrypted transaction dataset $\mathbf{D} = \{\vec{D}_1, \ldots, \vec{D}_m\}$, where $\vec{D}_j = ([\![d_{j,1}]\!]_{pk_{DO}}, \ldots, [\![d_{j,n}]\!]_{pk_{DO}})$, $j \in (1, \ldots, m)$. Before running our mining scheme, CSP sends m to Evaluator. Evaluator calculates $min \leftarrow \lceil supp_{\min} \cdot m \rceil$, and encrypted it as $[\![min]\!]$ before sending it to CSP. In our setting, we assume that the outsourced dataset is boolean, meaning that 1 denotes this item in this transaction otherwise it is not. As the example shown in Fig 2, the first transaction contains "Bread, Coke and Milk". Here, we stress that if the dataset is horizontally partitioned, the items in $\vec{D}_j$ are encrypted by the same public key, and different $\vec{D}_j$ may be encrypted by different keys. If the dataset is vertically partitioned, the items in $\vec{D}_j$ are encrypted by different public keys. No matter in which cases, our SFM could work successfully since we have designed a set of secure computation algorithms which can deal with data encrypted by different public keys. Before presenting our SFM, we first propose two sub-algorithms, i.e., Secure Find_frequent_1_itemsets (SF1I) and Secure Support Counting (SSC).

Secure Find_Frequent_1_Itemsets (SF1I): Suppose that CSP and Evaluator have an additive shared transaction dataset $\langle \mathbf{D} \rangle$. After running this algorithm, CSP gets the frequent_1_itemsets.

From line 1 to 5, CSP and Evaluator run Add on every column of our transaction dataset thus getting the $\langle s_i \rangle$. Then, they convert it as $[\![s_i]\!]_{pk_\Sigma}$. After that, CSP and Evaluator run ESC on $[\![s_i]\!]_{pk_\Sigma}$ and $[\![min]\!]_{pk_\Sigma}$. If the comparison result shows that $s_i \geq \min$, l_i is a frequent 1-itemset. Details of this SF1I are presented in Algorithm 2.

Algorithm 2: Secure Find_Frequent_1_Itemsets (SF1I).

Input: Transactions data $\langle \mathbf{D} \rangle = \{\langle \vec{D}_1 \rangle, \ldots, \langle \vec{D}_m \rangle\}$, where $\langle \vec{D}_j \rangle = (\langle d_{j,1} \rangle, \ldots, \langle d_{j,n} \rangle)$, $j \in [1, m]$.
Output: CSP gets all the frequent_1_itemsets.
1: Set $S_1 = (s_1, s_2, \ldots, s_n)$, where $s_i = 0$.
2: **for all** $i = 1$ to n **do**
3: $\langle s_i \rangle \leftarrow \texttt{Add} \sum_{j=1}^{m} \langle d_{j,i} \rangle$,
4: $[\![s_i]\!]_{pk_\Sigma} \leftarrow \texttt{S2L}(\langle s_i \rangle)$.
5: **end for**
6: Set $L_1 = \emptyset$.
7: **for all** $i = 1$ to n **do**
8: $t \leftarrow \texttt{ESC}([\![s_i]\!]_{pk_\Sigma}, [\![min]\!]_{pk_\Sigma})$.
9: **if** $t = 1$ **then**
10: $L_1 = L_1 \cup l_i$.
11: **end if**
12: **end for**
13: **return** L_1

Algorithm 3: Secure Support Counting (SSC).

Input: Dataset $\langle \mathbf{D} \rangle = \{\langle \vec{D}_1 \rangle, \ldots, \langle \vec{D}_m \rangle\}$, where $\langle \vec{D}_j \rangle = (\langle d_{j,1} \rangle, \ldots, \langle d_{j,n} \rangle)$, $j \in (1, \ldots, m)$. Itemsets $\langle \vec{G} \rangle = (\langle g_1 \rangle, \langle g_2 \rangle, \ldots, \langle g_n \rangle)$, and $[\![\lambda]\!]_{pk_M}$.
Output: CSP gets support of $\vec{G}$.
1: Set $c = 0$
2: **for all** every $\vec{D}_j$ **do**
3: $\langle e_j \rangle \leftarrow \texttt{SIP}(\langle \vec{G} \rangle, \langle \vec{D}_j \rangle)$.
4: $[\![e_j]\!]_{pk_\Sigma} \leftarrow \texttt{S2L}(\langle e_j \rangle)$.
5: $[\![t_j]\!]_{pk_\Sigma} \leftarrow \texttt{SCD}([\![e_j]\!]_{pk_\Sigma}, [\![\lambda]\!]_{pk_M})$.
6: $[\![c]\!]_{pk_\Sigma} \leftarrow [\![c]\!]_{pk_\Sigma} \cdot [\![t_j]\!]_{pk_\Sigma}$.
7: **end for**
8: **return** $[\![c]\!]$

Secure Support Counting (SSC): After running SSC, CSP obtains the encrypted support of some itemset $G = (g_1, g_2, \ldots, g_n)$. Here, λ is the number of 1 s in $\vec{G}$.

Specifically, CSP runs SIP with Evaluator on every $\langle \vec{D}_j \rangle$ and $\langle \vec{G} \rangle$. After that, they convert $\langle e_j \rangle$ to $[\![e_j]\!]_{pk_\Sigma}$. Then, CSP and Evaluator run SCD on $[\![e]\!]_{pk_\Sigma}$ and $[\![\lambda]\!]_{pk_M}$, thus obtaining the comparison result $[\![t_j]\!]$. CSP multiplies all the $[\![t_j]\!]$, and the multiplication result $[\![c]\!]$ is the encrypted support of $\vec{G}$. Details of this Secure Support Counting (SSC) are presented in Algorithm 3.

In the following, we present an example to illustrate frequent itemset mining and association rule mining. Here, we give a transaction dataset $\mathbf{D}$ shown in Fig. 2. Normally, all these items are represented as boolean types. Assume that the itemsets are "Coke, Milk". Then it can represented as $\vec{G} = (0,1,1,0)$. The CSP and Evaluator make an inner product $e_j = \vec{G} \cdot \vec{D}_j$, where $\vec{D}_j (j \in (1, \ldots, 5))$ is one of the row in the table. Then, miner

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4: Secure Frequent Itemset Mining (SFM).

Input: Dataset $\mathbf{D} = \{\vec{D}_1, \ldots, \vec{D}_m\}$, where $\vec{D}_j = (\llbracket d_{j,1} \rrbracket_{pk_{DO}}, \ldots, \llbracket d_{j,n} \rrbracket_{pk_{DO}})$, $j \in (1, \ldots, m)$.
Output: CSP gets all the frequent itemsets.

1: For each $\llbracket d_{i,j} \rrbracket$ in $\mathbf{D}$, $\langle d_{i,j} \rangle \leftarrow \texttt{L2S}(\llbracket d_{i,j} \rrbracket)$.
2: $L_1 = \texttt{SF1I}(D)$.
3: **for all** $k = 2; L_{k-1} \neq \emptyset; k++$ **do**
4: $C_k = \mathbf{apriori_gen}(l_{k-1})$.
5: **for all** every $C_{i,k} \in C_k$ **do**
6: CSP splits $C_{i,k}$ into additive shares, and sends $\langle C_{i,k} \rangle^B$ to Evaluator.
7: $\llbracket c \rrbracket_{pk_\Sigma} \leftarrow \texttt{SSC}(\langle C_{i,k} \rangle)$.
8: $t' \leftarrow \texttt{ESC}(\llbracket c \rrbracket_{pk_\Sigma}, \llbracket min \rrbracket_{pk_\Sigma})$.
9: If $t' = 1$, put $C_{i,k}$ into L_k.
10: **end for**
11: **end for**
12: **return** $L = \cup_k L_k$

Algorithm 5: Secure Association Rule Mining (SAM).

Input: Dataset $\langle \mathbf{D} \rangle = \{\langle \vec{D}_1 \rangle, \ldots, \langle \vec{D}_m \rangle\}$, where $\langle \vec{D}_j \rangle = (\langle d_{j,1} \rangle, \ldots, \langle d_{j,n} \rangle)$, $j \in (1, \ldots, m)$. The threshold confidence is α/β. The frequent itemsets is $\vec{f}$.
Output: CSP gets the strong association rules.

1: CSP splits α, β into two additive shares and sends $\langle \alpha \rangle^B, \langle \beta \rangle^B$ to Evaluator.
2: $\langle c \rangle \leftarrow \texttt{L2S}(\llbracket c \rrbracket_{pk_\Sigma})$, $\langle \tau_i \rangle \leftarrow \texttt{Mul}(\langle \beta \rangle, \langle c \rangle)$.
3: Get the sets of $\vec{f}$'s nonvoid proper subset H, where $H = \{\vec{h}_1, \ldots, \vec{h}_\theta\}$.
4: **for all** $\vec{h}_i \in H$ **do**
5: $\langle c_i \rangle \leftarrow \texttt{L2S}(\llbracket c_i \rrbracket_{pk_\Sigma})$.
6: $\langle \varepsilon_i \rangle \leftarrow \texttt{Mul}(\langle \alpha \rangle, \langle c_i \rangle)$, $\llbracket \varepsilon_i \rrbracket \leftarrow \texttt{S2L}(\langle \varepsilon_i \rangle)$.
7: $t_i \leftarrow \texttt{ESC}(\llbracket \tau_i \rrbracket_{pk_\Sigma}, \llbracket \varepsilon_i \rrbracket_{pk_\Sigma})$.
8: **if** $t_i = 1$ **then**
9: **return** $\vec{h}_i \Rightarrow (\vec{f} - \vec{h}_i)$.
10: **end if**
11: **end for**

compares e_j with λ, which is the number of 1 s in $\vec{G}$. Suppose that there is a c, which is used to count the frequency of itemsets "Coke, Milk". If $e_j ==$ λ, set $c = c + 1$. After the calculation, we can get the frequency of "Coke, Milk", i.e., c.

Secure Frequent Itemset Mining (SFM): In our SFM, we use Apriori as the basic mining algorithm. Running this SFM, the CSP gets all the frequent itemsets of D. The specific information of SFM can be found in Algorithm 4.

First, the CSP converts the outsourced encrypted data into additive secret shares with the help of the Evaluator. Second, CSP mines all the frequent 1-itemsets with Evaluator, which is shown in line 2. Then our algorithm iterates the following steps until all the frequent itemsets are found. CSP runs $\mathbf{apriori_gen}(l_{k-1})$ to get all the candidate frequent k-itemsets, which are denoted as $C_{i,k}$. CSP splits $C_{i,k}$ into secret shares, and sends $\langle C_{i,k} \rangle^B$ to Evaluator. CSP and Evaluator run SSC on every $\langle C_{i,k} \rangle$ to count their frequencies $\llbracket c \rrbracket_{pk_\Sigma}$. Note that, in this SSC, since λ can be encrypted by pk_Σ, the SCD can be substituted as SC. Then CSP and Evaluator run ESC on $\llbracket c \rrbracket_{pk_\Sigma}$ and $\llbracket \min \rrbracket_{pk_\Sigma}$. $C_{i,k}$ is a frequent k-itemset only if the comparison result is 1.

C. Secure Association Rule Mining

After running SFM, CSP gets all the frequent itemsets in plaintext. Based on these, CSP can easily get strong association rules. Assume that the minimum confidence is α/β. For every frequent itemset $\vec{f}$, CSP first gets all the nonvoid proper subset H. For example, if $\vec{f} = \{1, 1, 1, 0\}$ whose corresponding itemsets are $\{X, Y, Z\}$. Let the support of $\vec{f}$ be c/m. Note that, c has been calculated in SFM. The sets of $\vec{f}$'s nonvoid proper subset is

$$H = \{\{X\}, \{Y\}, \{Z\}, \{X, Y\}, \{X, Z\}, \{Y, Z\}\},$$

which can be represented as $H = \{\{1, 0, 0, 0\}, \{0, 1, 0, 0\}, \{0, 0, 1, 0\}, \{1, 1, 0, 0\}, \{1, 0, 1, 0\}, \{0, 1, 1, 0\}\}$. Note that, since $\vec{f}$ is a frequent itemset, all its nonvoid proper subset is frequent itemset too. Thus, the support of $\vec{f}$, i.e., $\llbracket c \rrbracket$, and its proper subset, i.e.,$\llbracket c_i \rrbracket$, have been calculated in SFM. CSP and Evaluator just need to transform them into additive shares. Then, they can run Mul on $(\langle \beta \rangle, \langle c \rangle)$ and $(\langle \alpha \rangle, \langle c_i \rangle)$ respectively, to obtain $\langle \tau_i \rangle$ and $\langle \varepsilon_i \rangle$ (Note that, if $c/c_i \geq \alpha/\beta$, $c \cdot \beta \geq c_i \cdot \alpha$). Next, they transform $\langle \tau_i \rangle$, $\langle \varepsilon_i \rangle$ into $\llbracket \tau_i \rrbracket$, $\llbracket \varepsilon_i \rrbracket$ and run ESC on them. If the comparison result is 1, $\vec{h}_i \Rightarrow (\vec{f} - \vec{h}_i)$ is a strong association rule. Here, $\vec{f} - \vec{h}_i$ is the bit vector representing the set of items that are in $\vec{f}$ but not in $\vec{h}_i$. Our Secure Association Rule Mining is given in Algorithm 5. CSP iterates these steps on all the mined frequent itemsets to get strong association rules. Note that, the clouds run SAM after running SFM. Therefore, CSP and Evaluator already have an additively shared dataset.

D. Secure Frequent Itemset Query

In our system, Miner, who has a limited transaction dataset and computation ability, is a resource-constrained user. To find out whether an itemset is frequent or not, he should send it to CSP rather than mining on his own little transaction dataset. In our system, the cloud provides secure frequent itemsets query service on mined frequent itemsets. There are two service models provided. In the first model, the miner can decide the threshold of support and the cloud will return the mining result according to this threshold. In the second one, the cloud sends the miner the query result according to his mined results from SFM.

CASE I: In this case, the Miner has minimum support $s_{\min}$ decided by himself, where $s_{\min} = s_1/s_2$. At first, Miner sends the encrypted query and threshold to CSP. Then, CSP and Evaluator transform them into additive shares. Then, CSP and Evaluator cooperate to run SSC on the query to get the frequency of $\vec{q}$. Moreover, they calculate $\langle m \cdot s_1 \rangle$ and $\langle s_2 \cdot c_q \rangle$, and transform them into ciphertext of LCS cryptosystem. In addition, they run SC on $\llbracket m \cdot s_1 \rrbracket_{pk_M}$ and $\llbracket s_2 \cdot c_q \rrbracket_{pk_M}$. And then CSP sends it to Miner. Miner decrypts it, thus getting the query result. Note that, $s_1/s_2 \geq c_q/m$, if and only if $s_1 \cdot m \geq s_2 \cdot c_q$. Our Secure Frequent Itemset Query I (SFQI) is shown as follows.

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

Step 1 (Miner): Send encrypted query $[\![\vec{q}]\!]_{pk_M} = ([\![q_1]\!]_{pk_M}, [\![q_2]\!]_{pk_M}, \ldots, [\![q_n]\!]_{pk_M})$, $[\![s_1]\!]_{pk_M}$ and $[\![s_2]\!]_{pk_M}$ to CSP.

Step 2 (CSP & Evaluator): (1) Transform the encrypted query into additive shares, $\langle \vec{q} \rangle \leftarrow \texttt{L2S}([\![\vec{q}]\!]_{pk_M})$. Calculate the frequency of $\vec{q}$ in the dataset by running SSC, i.e., $[\![c_q]\!]_{pk_\Sigma} \leftarrow \texttt{SSC}(\langle \vec{q} \rangle)$.

(2) Transfrom $[\![s_1]\!]_{pk_M}$, $[\![s_2]\!]_{pk_M}$ and $[\![c_q]\!]_{pk_\Sigma}$ into additive shares, i.e., $\langle s_1 \rangle \leftarrow \texttt{L2S}([\![s_1]\!]_{pk_M})$, $\langle s_2 \rangle \leftarrow \texttt{L2S}([\![s_2]\!]_{pk_M})$, and $\langle c_q \rangle \leftarrow \texttt{L2S}([\![c_q]\!]_{pk_\Sigma})$. Then, calculate $\langle s'_2 \rangle \leftarrow \texttt{Mul}(\langle s_2 \rangle, \langle c_q \rangle)$, $\langle s'_1 \rangle \leftarrow m \cdot \langle s_1 \rangle$.

(3) Transform $\langle s'_1 \rangle$ and $\langle s'_2 \rangle$ into ciphertext encrypted by Miner's public key, i.e., $[\![s'_1]\!]_{pk_M} \leftarrow \texttt{S2L}(\langle s'_1 \rangle)$, $[\![s'_2]\!]_{pk_M} \leftarrow \texttt{S2L}(\langle s'_2 \rangle)$. Then, make a comparison with them, $[\![\zeta]\!]_{pk_M} \leftarrow \texttt{SC}([\![s'_1]\!]_{pk_M}, [\![s'_2]\!]_{pk_M})$. Finally, send $[\![\zeta]\!]_{pk_M}$ to Miner.

Step 3 (Miner): Decrypt $[\![\zeta]\!]_{pk_M}$. If $\zeta = 1$, $\vec{q}$ is a frequent itemset, else it is not.

REMARK. In the previous works [1], [37], [18], the dummy transactions are added to conceal the real support of the query. In those works, to get the support of the query, the Evaluator must decrypt the comparison result of the inner product between a transaction with the query. Without the dummy transactions, the support of the query may be leaked to the Evaluator. While, in this work, our proposed SSC can be used to calculate the support of the query in ciphertext by Evaluator.

CASE II: In this case, we assume Miner wants to find out whether the query is frequent or not according to CSP's mining results.

First, Miner sends the encrypted query $[\![\vec{q}]\!]_{pk_M}$ and $[\![num_q]\!]_{pk_M}$ to CSP, where num_q is the number of 1 s in $\vec{q}$. CSP and Evaluator convert $[\![\vec{q}]\!]_{pk_M}$ and $[\![num_q]\!]_{pk_M}$ to additive secret shares. Moreover, CSP also splits all the frequent itemsets into additive shares and sends one of the shares to Evaluator. Then, they run SIP on $\vec{q}$ and every frequent itemsets $\vec{f_i}$. After that, they calculate $\varrho_i = \omega_i - num_q$, through running Add on $\langle \omega_i \rangle$ and $\langle -num_q \rangle$. CSP blinds ϱ_i with a random number α_i, and then converts it to the data encrypted by Miner's public key. CSP randomly chooses several numbers β_j in $\mathbb{Z}_N$, where $j \in (1, 2, \ldots, p)$, and encrypts them by pk_M. CSP runs a permutation on $\{[\![\varrho'_1]\!]_{pk_M}, [\![\varrho'_2]\!]_{pk_M}, \ldots, [\![\varrho'_k]\!]_{pk_M}\}$, before sending them to Miner. The miner decrypts the received data. If one of them is 0, the query vector is frequent. Otherwise, it is not. Details of the Secure Frequent Itemset Query II) (SFQII) algorithm are shown in Algorithm 6.

REMARK. The key problem of our SFQ is to find out whether $\vec{q}$ is in the frequent itemsets $\vec{F}$. $\vec{q}$ and $\vec{f_i}$ are all binary vectors. we make an inner product of them. If the result is equal to the 1 s in $\vec{q}$, it is obvious that $\vec{q}$ is equal to one of $\vec{f_i}$. In this algorithm, we blind every ϱ_i with a random number to avoid Miner inferring some information about CSP's mining results.

Discussion: Here, we discuss a method to accelerate our algorithm. If Miner sends his query $\vec{q}$ in ciphertext while num_q in plaintext, the query process will be more efficient but with a lower privacy level. Let's assume that the largest frequent itemsets mined from SFM is n_c. If $num_q > n_c$, CSP can tell Miner the query itemsets are not frequent right now. If $num_q \leq n_c$, CSP just needs run SIP on $\vec{q}$ and all the mined frequent num_q-itemsets rather than all the frequent itemsets.

Algorithm 6: Secure Frequent Itemset Query II (SFQII).

Input: Miner has the encrypted query $[\![\vec{q}]\!] = ([\![q_1]\!]_{pk_M}, [\![q_2]\!]_{pk_M}, \ldots, [\![q_n]\!]_{pk_M})$. CSP has all the frequent itemset $\vec{F} = \{\vec{f_1}, \vec{f_2}, \ldots, \vec{f_k}\}$.

Output: Miner gets the query result.

1: Miner sends encrypted query $[\![\vec{q}]\!] = ([\![q_1]\!]_{pk_M}, [\![q_2]\!]_{pk_M}, \cdots, [\![q_n]\!]_{pk_M})$ and $[\![num_q]\!]_{pk_M}$ to CSP, where num_q is the number of 1 s in $\vec{q}$.
2: CSP and Evaluator calculate $\langle num_q \rangle \leftarrow \texttt{L2S}([\![num_q]\!]_{pk_M})$, $\langle \vec{q} \rangle \leftarrow \texttt{L2S}([\![\vec{q}]\!]_{pk_M})$.
3: CSP splits $\vec{F}$ into two additive shares and sends $\langle \vec{F} \rangle^B$ to Evaluator.
4: **for all** $\vec{f_i} \in \vec{F}$ **do**
5: CSP and Evaluator calculate $\langle \omega_i \rangle \leftarrow \texttt{SIP}(\langle \vec{q} \rangle \cdot \langle \vec{f_i} \rangle)$ and $\langle \varrho_i \rangle \leftarrow \texttt{Add}(\langle \omega_i \rangle, \langle -num_q \rangle)$.
6: Randomly choose a public constant α_i from $\mathbb{Z}^*_{N'}$. Then calculate $\langle \varrho'_i \rangle^A \leftarrow \alpha_i \cdot \langle \varrho_i \rangle^A$, $\langle \varrho'_i \rangle^B \leftarrow \alpha_i \cdot \langle \varrho_i \rangle^B$. Transform it into ciphertext, i.e., $[\![\varrho'_i]\!]_{pk_M} \leftarrow \texttt{S2L}(\langle \varrho'_i \rangle)$.
7: **end for**
8: CSP runs a permutation function on $\{[\![\varrho'_1]\!]_{pk_M}, [\![\varrho'_2]\!]_{pk_M}, \cdots, [\![\varrho'_k]\!]_{pk_M}\}$, before sending them to Miner.
9: Miner decrypts all the received data. If one of them is equal to 0, it is frequent. Otherwise, it is not.

Under this condition, the number of 1 s in the query is leaked to CSP.

E. Secure Association Rule Query

If the query in SFQ is a frequent itemset, Miner may query CSP for the strong association rules of this frequent itemset. In this section, we assume that Miner wants to find out whether $\vec{A} \Rightarrow \vec{B}$ is a strong association rule or not, where $\vec{A}$ and $\vec{B}$ are itemsets. Here, CSP also provides two service models as in SFQ.

Case I: If Miner has a minimum confidence (i.e., γ/δ) decided by himself, our Secure Association Rule Query I (SAQI) is shown as follows.

Step 1 (Miner): Send encrypted query $[\![\vec{A}]\!]_{pk_M}$, $[\![\vec{A} \cup \vec{B}]\!]_{pk_M}$, $[\![\gamma]\!]_{pk_M}$ and $[\![\delta]\!]_{pk_M}$ to CSP.

Step 2 (CSP & Evaluator): Transform the ciphertext of $[\![\vec{A}]\!]_{pk_M}$ and $[\![\vec{A} \cup \vec{B}]\!]_{pk_M}$ into additive shares by running L2S. Then, run SSC on $\langle \vec{A} \rangle$ and $\langle \vec{A} \cup \vec{B} \rangle$, where $c_{\vec{A}}$ and c are the frequencies of $\vec{A}$ and $\vec{A} \cup \vec{B}$.

Step 3 (CSP & Evaluator): Transform $[\![\gamma]\!]_{pk_M}$, $[\![\delta]\!]_{pk_M}$, $[\![c_{\vec{A}}]\!]_{pk_\Sigma}$ and $[\![c]\!]_{pk_\Sigma}$ into additive shares, by running L2S, i.e., $\langle \gamma \rangle \leftarrow \texttt{L2S}([\![\gamma]\!]_{pk_M})$, $\langle \delta \rangle \leftarrow \texttt{L2S}([\![\delta]\!]_{pk_M})$; $\langle c_{\vec{A}} \rangle \leftarrow \texttt{L2S}([\![c_{\vec{A}}]\!]_{pk_\Sigma})$, $\langle c \rangle \leftarrow \texttt{L2S}([\![c]\!]_{pk_\Sigma})$.

Step 4 (CSP & Evaluator): Calculate $\delta \cdot c_{\vec{A}}$, $\gamma \cdot c$ i.e., $\langle \phi \rangle \leftarrow \texttt{Mul}(\langle \delta \rangle, \langle c_{\vec{A}} \rangle)$, $\langle \varphi \rangle \leftarrow \texttt{Mul}(\langle \gamma \rangle, \langle c \rangle)$. Transform the additive shares into LCS ciphertext by running S2L, i.e, $[\![\phi]\!]_{pk_M} \leftarrow \texttt{S2L}(\langle \phi \rangle)$, $[\![\phi]\!]_{pk_M} \leftarrow \texttt{S2L}(\langle \varphi \rangle)$. Then run SC on

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

them, $[\![\xi]\!]_{pk_M} \leftarrow \mathtt{SC}([\![\phi]\!]_{pk_M}, [\varphi]\!]_{pk_M})$. Finally, send $[\![\xi]\!]_{pk_M}$ to Miner.

Step 5 (Miner): Decrypt $[\![\xi]\!]_{pk_M}$. If $\xi = 1$, it is a strong association rule, else it is not.

Case II: In this case, Miner wants to know whether $\vec{A} \Rightarrow \vec{B}$ is a strong association rule or not, according to CSP's minimum confidence. Suppose there are η mined strong association rules in CSP, and they can be denoted as $\vec{C}_i \Rightarrow \vec{D}_i$, where $i \in (0, \ldots, \eta)$. To solve this problem, we propose a Secure Association Rule Query II (SAQII) algorithm, which is quite similar to our SFQII. Here, we denote $\vec{C}_i$ and $\vec{D}_i$ as a binary vector respectively and then combine them all together. For instance, if $\vec{C}_i = \{X, Y\}, \vec{D}_i = \{Z\}$, we can represent $\vec{C}_i \Rightarrow \vec{D}_i$ as (1,1,0,0,0,0,1,0), where (1,1,0,0) is $\vec{C}_i$ and (0,0,1,0) is $\vec{D}_i$. Then, our SAQII goes the same steps as in our SFQII. First, they query whether $\vec{A}$ is one of $\vec{C}_i, i \in [1, \eta]$ or not. If it is not, return the Miner 0, meaning that this query is not a strong association rule. Else, they query whether $\vec{A} \cup \vec{B}$ in one of $\vec{C}_i \cup \vec{D}_i, in \in [1, \eta]$.

VI. Security Analysis

A. Security Analysis of the Cryptographic Blocks

The security analysis of the proposed building blocks is presented in this section. Before that, we introduce the security definition used in the semi-honest model [18], [38].

Definition 4 (Security in the Semi-Honest Model [38]): Suppose that party P_i's input is a_i, P_i's execution image of protocol π is $\Pi_i(\pi)$ and P_i's computation output from π is b_i. Thus, π is considered to be secure if the distribution of its simulated image $\Pi_i^S(\pi)$ from a_i and b_i cannot be computationally distinguishable from $\Pi_i(\pi)$ (Specific details are shown in [38]).

According to *Definition 4*, it can be concluded that if a cryptographic building block is secure, its execution image should be computationally indistinguishable from its simulated image. Generally, we stress that the data exchanged and the information calculated from the protocol are all included in the execution image.

Lemma 1: If all the subprotocols are perfectly simulatable, a protocol is perfectly simulatable [39].

More details of the proof about Lemma 1 can be found in [39].

Theorem 1: The S2L and L2S are secure under the semi-honest model.

Proof 1: First, in our S2L, the Evaluator sends the encrypted share to CSP while CSP multiplies it with his own encrypted share. Since CSP does not have the decryption key, we can conclude that S2L is secure. Next, in our L2S, the execution image of Evaluator is $\Pi_{Evaluator}(\mathtt{L2S}) = \{Y, y\}$. Note that $y = x - \alpha$, where y is random because of the randomness of α. We also assume that the simulated image of Evaluator is $\Pi^S_{Evaluator}(\mathtt{L2S}) = \{Y', y'\}$, where Y' is pair of data randomly picked from $\mathbb{Z}^*_{N^2}$ and y' is a random number from $\mathbb{Z}^*_N$. Since LCS is semantically secure under $\mathbb{Z}^*_{N^2}$ [21], we can conclude that $\Pi_{Evaluator}(\mathtt{L2S})$ is computationally indistinguishable from $\Pi^S_{Evaluator}(\mathtt{L2S})$. ■

Theorem 2: The SIP is secure under the semi-honest model.

Proof 2: Our SIP is only based on Add and Mul, whose security have been proved in [25] and [26]. Thus, it can be concluded that the designed SIP is secure. ■

Theorem 3: The SCD is secure under the semi-honest model.

Proof 3: First, we denote the execution image for the Evaluator of our SCD as $\Pi_{Evaluator}(\mathtt{SCD})$, where $\Pi_{Evaluator}(\mathtt{SCD}) = \{(C', c'), (D', d'), (F', z), \delta\}$. We stress that $c' = (-1)^a \cdot (2x + 1) + r_a$, $d' = (-1)^{1-a} \cdot (2y) + r_b$, $z = r_1((-1)^a \cdot (2x + 1) + (-1)^{1-a} \cdot (2y)) + r_2$, which are the decryption of C', D', F'. Since a is a random bit chosen from (0,1) and r_a, r_b are randomly chosen from $\mathbb{Z}^*_N$, thus c' and d' are uniformly distributed over $\mathbb{Z}^*_N$. Moreover, $r_1 \ll r_2$ are randomly picked from $\{1, \ldots, 2^l\}$ where $2^{2l+1} < N/2$, and δ is the comparison result which is 0 or 1. Next, we represent the simulated image for the Evaluator of our SCD as $\Pi^S_{Evaluator}(\mathtt{SCD}) = \{(C'', c''), (D'', d''), (F'', z'), \delta'\}$ where (C'', D'', F'') (resp. (c'', d'', z')) are randomly generated from $\mathbb{Z}^*_{N^2}$ (resp. $\mathbb{Z}^*_N$), and δ' is set to 1 or 0 with equal probability. Since LCS is semantically secure, (C', D', F') are computationally indistinguishable from random values from $\mathbb{Z}^*_{N^2}$, meaning that (C', C''), (D', D''), (F', F'') are computationally indistinguishable. Furthermore, we can also infer that (c', c''), (d', d''), (z, z') and (δ, δ') are also computationally indistinguishable. Combining the above analyses, we can draw a conclusion that $\Pi_{Evaluator}(\mathtt{SCD})$ is computationally indistinguishable from $\Pi^S_{Evaluator}(\mathtt{SCD})$.

With the similar proof skill, we can also prove that $\Pi_{CSP}(\mathtt{SCD})$ is computationally indistinguishable from $\Pi^S_{CSP}(\mathtt{SCD})$. First, the execution image of CSP can be denoted by $\Pi_{CSP}(\mathtt{SCD})$, in which $\Pi_{CSP}(\mathtt{SCD}) = \{E, [\![\delta]\!]_{pk_\Sigma}\}$. We also represent the simulated image of CSP as $\Pi^S_{CSP}(\mathtt{SCD}) = \{E', [\![\delta]\!]'_{pk_\Sigma}\}$, in which two elements are randomly picked from $\mathbb{Z}^*_{N^2}$. Since LCS is semantically secure, we can infer that $\Pi_{CSP}(\mathtt{SCD})$ is computationally indistinguishable from $\Pi^S_{CSP}(\mathtt{SCD})$.

Based on the above analysis, we can claim that SCD is secure under the semi-honest model.

Similar to the proofs in Theorem 3, we can easily conclude that SC, ESCD, and ESC are secure. ■

B. Security of SFM, SAM, SFQ and SAQ

Theorem 4: The SF1I and SSC are secure under the semi-honest model.

Proof 4: First, in SF1I algorithm, CSP and Evaluator use Add to count the support of every itemset and convert the shared support to the LCS ciphertext by S2L. Then, they use ESCD to compare it with $supp_{\min}$. Next, in SSC algorithm, CSP and Evaluator run SIP to calculate the inner product of $\langle \vec{G} \rangle$ and $\langle \vec{D}_j \rangle$, then convert it to LCS ciphertext. Finally, CSP and Evaluator run SCD on two encrypted values. Since we have stated the security of Add, S2L, ESCD, SIP, and SCD under the semi-honest model, according to Lemma 1 we can conclude that our SF1I and SSC are secure under the semi-honest model. ■

Theorem 5: The SFM and SAM are secure under the semi-honest model and also could protect the original outsourced data's security from the outside adversary $\mathcal{A}$.

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

Proof 5: Our SFM is based on L2S, SF1I, SSC and ESCD. Moreover, our SAM is based on L2S, Mul, S2L and ESCD. Since we have proved that all of them are secure under the semi-honest model, we could draw a conclusion that the designed SFM and SAM are secure under the semi-honest model according to Lemma 1.

Suppose that $\mathcal{A}$ can eavesdrop on the communication channels between CSP and Evaluator, $\mathcal{A}$ then all obtain the intermediate data transmitted. However, they are encrypted by a semantically secure cryptosystem, $\mathcal{A}$ is not able to derive the plaintext without the decryption key. Note that, even if the Evaluator is compromised, our scheme is also secure because all the data received by Evaluator is blinded with random numbers.

As a result, we can claim that our SFM and SAM are secure under the semi-honest model and also can preserve the original outsourced data confidentiality against the outside adversary. ■

By the same argument, we also have that the simulation for our SFQI, SFQII, SAQI and SAQII are indistinguishable from a real execution on the semi-honest model. We give the following theorem.

Theorem 6: The SFQI, SFQII, SAQI and SAQII are secure under the semi-honest model and also could protect the original outsourced data's security from the outside adversary $\mathcal{A}$.

Theorem 7: The SFQI, SFQII, SAQI and SAQII all achieve the Privacy Level IV. ■

Proof 6: From theorem 6 and the definition of the semi-honest model, it can be inferred that two clouds learn nothing about each other's input. Moreover, since the query data and mined results are all encrypted by the miner's public key where the secret key is not kept by two clouds, nothing of the query data and mined results are inferred by the CSP and Evaluator. Thus, the leakage function $\mathcal{L}$ of these algorithms can be represented as $\mathcal{L}(\kappa, \vec{x}) = \{\emptyset\}$, where κ is the security parameter and $\vec{x}$ denotes the input data. With the above analyses, we can conclude that SFQI, SFQII, SAQI and SAQII all achieve the Privacy Level IV.

VII. Performance Analysis

A. Experiment Analysis

The performance evaluations of the proposed system are tested on several computers. The two clouds are simulated by two personal computers running Ubuntu 18.04.2 LTS with Intel Core I7-6700 3.4 GHz CPU and 16 GB RAM. The others are the data owners which are tested on two laptop computers running Ubuntu 18.04.2 LTS with Intel Core I5-5200 U 2.20 GHz CPU and 4 GB RAM. We implement LCS cryptosystem by C code with the GNU MP library[2] and use this to implement our computation protocols.

1) Efficiency of Our Algorithms: For the LCS cryptosystem, we denote N as 1024 bits to achieve 80-bit security [40] levels. Note that the bit-length of the outsourced data and the calculated intermediate integers are less than 64. Thus, in our test we set $\|N'\| = 64$. Before running the basic cryptographic blocks, we test the LCS cryptosystems 1000 times. LCS takes 2.39 ms to run **Enc**, 1.06 ms to run **Dec** and 1.02 ms to run **mDec**. Generally, for all the algorithms, there are three main factors mainly deciding the performance, i.e., 1) the crypto (LCS) parameter N, 2) the length of a transaction n, and 3) the number of transactions m in a dataset.

We evaluate the performance of all the proposed algorithms with varied crypto (LCS) parameter N, which can be seen in Table II. From Table II, we can see that both the computation and communication costs of the proposed algorithms increase with N except SIP because the SIP is just based on secret sharing technology. In such a scheme, no encryption and decryption is needed.

In this section, we also test the CSP's and Evaluator's computation and communication costs in SFM, SAM, SFQI, SFQII, SAQI and SAQII with different m, n. The evaluation is based on the LCS cryptosystem with $\|N\| = 1024$. From Fig. 3(a) and (c), we can see that the computation and communication costs of SF1I nearly stay the same with different m. The reason behind it is that in our SF1I, the sub-algorithm whose computation is related with m is Add, which is made locally and needs little computation cost. The computation and communication costs of the other algorithms increase with m. Note that, from the description of Algorithm 6, it seems that the computation and communication costs of our SFQII and SAQII are not directly related to m. But the larger m will lead to more frequent itemsets and association rules. Thus, their costs in fact increase with m. From Fig. 3(b) and (d), the computation and communication costs of all the algorithms increase with n. Note that the computation and communication costs of SSC increase quite slowly with n, because, in SSC, SIP is the only sub-algorithm related to n, which is quite efficient. Moreover, we can see that the computation and communication costs of SFM do not increase with n strictly. The reason behind it is that when increasing n, the support of one item may be smaller. Thus, it will influence the computation and communication costs. Besides, in general, when increasing m and n, the computation and communication costs of SFM and SAM will grow, since it may lead to more frequent itemsets and association rules. Generally, The experimental results consistent with the theoretical analysis.

2) Real-World Dataset Experiment: In this section, we evaluate our system on two real-world datasets, i.e., chess, and mushroom,[3] as our transaction datasets. Our evaluation is made on the two PCs running Ubuntu 18.04.2 LTS with Intel Core I7-6700 3.4 GHz CPU and 16 GB RAM. We choose $\|N\| = 1024$ as the basic crypto parameter. Moreover, all the experiments are programmed with single-thread.

In the test of the chess dataset, which has 3196 transactions and 75 attributes, we set $supp_{\min} = 0.97$ and $conf_{\min} = 0.98$. In our SFM, the CSP and Evaluator need about 72.1 minutes to run and transmit about 455 MB of data. After running SFM, CSP gets 29 frequent itemsets and the largest frequent itemsets are frequent 4-itemsets. To get all the strong association rules, two clouds need to run SAM 29 times, since each SAM can just find one frequent itemsets' corresponding strong association rules. The total time and transmitted data are 5.19 s and 293.9 KB. To run

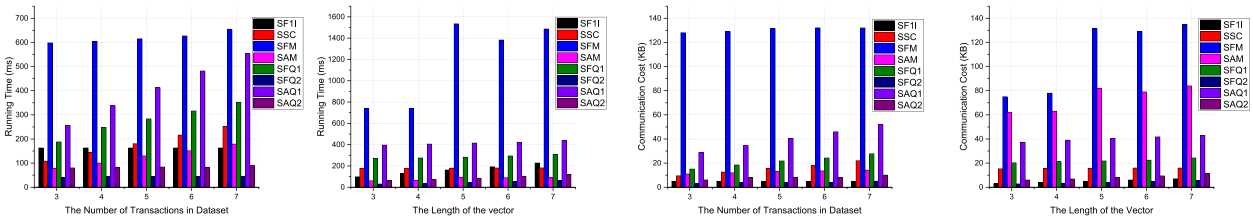
[2]https://gmplib.org/

[3]http://fimi.ua.ac.be/data/

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE OF OUR ALGORITHMS (1000-TIMES FOR AVERAGE, $m = n = 5$, $\tau = 7$, $\theta = 3$, $\rho = \rho' = 10$, $k = \iota = 2$)

Algorithm	Computation Costs(ms)							Communication Costs(KB)						
	512	768	1024	1280	1536	1792	2048	512	768	1024	1280	1536	1792	2048
S2L	1.21	2.47	5.06	8.92	13.66	20.65	29.92	0.25	0.375	0.5	0.625	0.75	0.875	1
L2S	1.06	1.94	3.91	6.75	10.33	15.58	22.53	0.25	0.375	0.5	0.625	0.75	0.875	1
SCD	5.71	12.13	29.74	46.56	80.65	119.84	171.76	1.25	1.875	2.5	3.125	3.75	4.375	5
ESCD	5.7	12.01	27.48	42.32	74.84	109.88	156.96	1	1.5	2	2.5	3	3.5	4
SC	3.02	6.24	18.32	29.3	42.35	71.92	97.54	0.5	0.75	1	1.25	1.5	1.75	2
ESC	3.01	6.22	18.01	25.01	36.49	61.83	82.27	0.25	0.375	0.5	0.625	0.75	0.875	1
SIP	1.27	1.24	1.25	1.25	1.25	1.26	1.25	0.156	0.156	0.156	0.156	0.156	0.156	0.156
SF1I	34.55	72.4	162.7	256.2	442.5	652.65	934.4	2.5	3.75	5	6.25	7.5	8.75	10
SSC	40.95	78	180.25	283.65	477.8	708.75	1014.6	8.281	12.03	15.78	19.53	23.28	27.03	30.78
SFM	387.6	751.6	1648	2391	4086	6436	8783	68.48	99.96	131.46	162.96	194.46	225.96	253.46
SAM	18.69	28.89	81.69	124.2	189.9	304	305	2.25	3.38	5.65	6.75	6.85	7.92	8.98
SFQI	58.29	126.04	283.35	461.98	757.91	1139.75	1613.9	11.28	16.53	21.78	27.031	32.28	37.53	42.87
SFQII	13.96	23.42	43.63	77.84	117.82	175.8	256.34	2.06	3.06	4.06	5.06	6.06	7.06	8.06
SAQI	92.75	178.82	412.4	653.64	1128.5	1649.8	2322.5	21.08	30.82	40.57	50.33	60.08	69.96	79.67
SAQII	26.4	46.01	86.91	154.8	134.3	350.2	512.6	4.11	6.12	8.13	10.11	12.14	14.19	16.12



(a) Runtime ($n = 5$) (b) Runtime ($m = 5$) (c) Communication Cost ($n = 5$) (d) Communication Cost($m = 5$)

Fig. 3. Performance of our algorithms.

TABLE III
PERFORMANCE COMPARISON FOR CHESS DATASET

Protocol	[19]'s protocol	[18]'s protocol based on BCP	[18]'s protocol based on BCP variant	SFQI
Running Time	2164.9 min	4321.6 min	2930.68 min	3.6 min

SFQI, CSP and Evaluator need 3.6 minutes for the computation while 16.8 MB data for the transmission. In this case, the Miner just needs 179 ms to encrypt the query and sends 37.5 KB of data. For the SFQII, two clouds take 1.08 s for the computation and transmit 132.4 KB data. In this case, the Miner takes 182.3 ms for the encryption and sends 37 KB of data to the cloud. For our SAQI, the clouds take 7.8 minutes for the computation and transmit 34.5 MB data for the communications, while the Miner takes 360 ms for the encryption and sends 74 KB data. For our SAQII, the clouds run 2.17 s for the computation and transmit 264.5 KB data. The Miner needs 364.9 ms for the encryption and sends 74 KB of data to the clouds.

Remark: Since this dataset has been used in some other datasets, we also present the performance comparison in Table III. From Table III, we can see that our SFQI outperform the most recent works such as [19] by 601 × and [18] by 813 × in clouds' running time.

Besides, we also test our system on a larger dataset, i.e., mushroom. For the test of the mushroom dataset, which has 8124 transactions and 119 attributes, we also set $supp_{\min} = 0.97$ and $conf_{\min} = 0.98$. In the SFM, CSP and Evaluator take 86.3 minutes to run and the data transmitted between them are 213.6 MB. After running this algorithm, CSP gets 7 frequent itemsets, and the largest frequent itemsets have 3 itemsets. Therefore, the two clouds need to run SAM 7 times in this dataset to find all the strong association rules. During this process, CSP and Evaluator take 339.4 ms for the computation and transmit 20 KB of data. In our SFQI, the two clouds take 8.96 min and transmit 53.58 MB data. In this case, the computation and communication cost of the Miner is 285.3 ms and 59.5 KB respectively. In the SFQII, the clouds need 730.5 ms for the computation. The data transmitted is 93.3 KB. In this case, Miner needs 475 ms for encryption and transmits 60 KB of data. For the SAQI, the clouds need 18.2 min for the computation and also transmit 108.2 MB data. The miner takes 988 ms for the encryption and also transmits 120 KB of data. Moreover, for the SFQII, computation and communication costs for the clouds (resp. Miner) are 36.9 minutes (resp. 955 ms) and 216.5 MB (resp. 120 KB) data. Note that, the mushroom dataset has nearly twice times many transactions as the chess dataset, but the running time is just a little more. The reason behind it is that the mushroom dataset is more sparse, and has less frequent itemsets.

From the description of our mining scheme and the query scheme, we can see that SSC is the most time-consuming part of our system. The larger the dataset, the more time CSP and Evaluator need for the computation. Note that in our system, SFM and SAM can be done offline, meaning that training time is not related to the quality of our SFQ or SAQ service. From the above test we can figure out that if a miner sends a query with his own decided threshold, the computation and communication

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON SUMMARY

Algorithm	Support FIM	Support ARM	Support FIQ	Support ARQ	Support Offline	Data Privacy	Ming Result Privacy	Client Cost	Support Multi-key
[41]	Yes	Yes	No	No	No	Yes	Yes	Low	No
[15]	Yes	No	No	No	No	Yes	Yes	High	No
[16]	No	Yes	No	No	Yes	Yes	Yes	High	No
[4]	Yes	Yes	No	No	Yes	Yes	No	Medium	No
[17]	Yes	Yes	No	No	Yes	Partial	Yes	Low	No
[42]	No	Yes	No	No	Yes	Yes	Yes	High	No
[1]	No	No	Yes	No	Yes	Yes	No	High	No
[18]	No	No	Yes	Yes	Yes	Yes	Yes	Medium	Yes
[36]	No	No	Yes	No	Yes	Yes	No	High	No
[19]	No	No	Yes	Yes	Yes	Yes	Yes	High	Yes
[20]	No	No	Yes	Yes	Yes	Yes	Yes	High	No
Ours	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Medium	Yes

cost will be much larger than querying the cloud for its trained result. Because if the query is with a new threshold, the CSP should query the whole dataset which costs too much time.

In addition, we also make an accuracy test of our scheme on the chess and mushroom datasets. The accuracy test results are shown in the Appendix, which can be found on the Computer Society Digital Library at http://doi.ieeecomputersociety.org/10.1109/TCC.2023.3269510.

B. Further Optimization Methods

Several methods for improving the efficiency of our mining and query process are illustrated in this section. First, multi-thread programming can be helpful. In our test, we only use sing-thread programming to evaluate our system. Adopting multi-thread programming can greatly help us decrease the running time of our system. Note that in our SFM, L2S and SSC are the most time-consuming parts. In fact, the loops of L2S and SSC can be paralleled, which may greatly improve the efficiency of SFM. We also made a new experiment on a chess dataset programmed with an 8-thread. The experimental results show that the clouds just need 18.62 minutes to run SFM. Since we can run our system with more threads on the real cloud, our scheme can be much more efficient in the real cloud. Besides, in real-world deployment, CSP and Evaluator will have much more powerful hardware. We use some quite common PC with Intel Core I7-6700 U CPU and 16 GB RAM to simulate our system. In a real-world deployment, the cloud would be deployed on workstations with more cores and using the most powerful GPUs as the computation cores.

C. Comparative Analysis

In this section, we introduce the comparison between our proposed scheme and the most related works designed recently. The details are shown in Table IV. In this table, for space limitation, we use FIM, ARM, FIQ, and ARQ to denote Frequent Itemset Mining, Association Rule Mining, Frequent Itemset Query, and Association Rule Query respectively. From Table IV, we can conclude that most of the recent works [41], [15], [16], [4], [17], [42] don't support frequent itemsets query and association rule query service. And few of the recent works support data owners and miners with multi-keys. Huang et al.'s work [16], [42] just supports association rule mining but cannot support frequent itemsets mining, and the data owners' computation cost is high. Dong et al.'s work [41], and Liu et al.'s work [15] cannot support offline users. Li et al.'s work [17] is quite efficient but it leaks partial raw data. There are three privacy-preserving frequent itemsets mining schemes with different privacy levels proposed in references [4] and [1]. In this section, our comparisons are made between their highest-level protocols with ours. In these two schemes [1] and [4], the miner's mining results cannot be well protected which is known by the cloud. Besides, the computational overhead of the data owner in Qiu et al.'s work [1] is high. In Liu et al.'s work [18], the scheme proposed in fact just supports frequent itemsets and association rules queries. In that scheme, a lot of dummy transactions are needed, which cost too much time for the clouds. Moreover, since our scheme supports offline users, the communication round between the miner with the cloud is just 1. Ma et al. improved Qiu et al.'s work [36]. Nevertheless, their work just improves the efficiency, but does not improve the security, and also cannot support SFM, SAM, and SAQ. Pang et al.'s scheme [19] and Chen et al.'s scheme [20] only support SFQ and SAQ. Moreover, since dummy transactions are needed to be added, the computation and communication costs of these schemes are high. Since the schemes of [18], [19], [20] can all protect the database uploaded from DOs, the query data and mined results of the Miner from the CSP and Evaluator, we can conclude that these schemes achieve Privacy Level IV too.

VIII. RELATED WORK

The research on privacy-preserving data mining can be categorized into randomization-based methods and cryptography-based methods. Randomization-based methods are based on data perturbation technology which usually can just protect the security of the data but without the mining results [6], [43], [44], [45]. Moreover, the accuracy may be different with non-privacy-preserving data mining methods.

Different from randomization-based methods, cryptography-based methods are designed under a well-defined security model, which can provide a higher level of data protection. Moreover, the mining results are usually the same as the non-privacy-preserving ones. Vaidya et al. first proposed an association rule mining scheme that can protect the data security [13]. However, this work is not efficient enough when dealing with large

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

datasets. Dong et al.'s designed a two-party privacy-preserving association rule mining method, where the two parties need many interactions [46]. The first semantically secure method for secure association rule mining is designed by Lai et al. [14]. Liu et al. [15] employ fully homomorphic encryption to build a frequent pattern mining scheme. Their scheme cannot support association rule mining and offline users. Huang et al. [16], [42] present a system that supports association rule mining but cannot support frequent itemsets mining. In addition, the data owners' computation costs in their scheme are high. The scheme for secure mining association rules on cloud data designed by Yi et al. involves $n(\geq 2)$ "semi-honest" servers. Since many clouds are required in their scheme, the communication costs are huge. For vertically divided datasets, Li et al. designed a cloud-assisted frequent itemsets and association rule mining system, which means the final mined results belong to the data owner other than the cloud. The motivating scenario is quite different from ours. Although their work is quite efficient, partial information of the raw data is leaked. Qiu et al. partially improved Yi et al.'s work by partially designing three different privacy level privacy-preserving frequent itemsets mining protocols [1]. However, the protocol cannot protect the query data and the mining result. The protocols II and III, the mining result is still revealed to the cloud. Another issue existing in protocol II is that the data owner's computation cost is large since using a time-consuming cryptosystem, i.e., BGN. Besides, their work just focuses on frequent itemset mining. They did not give a method to mine the association rule from the frequent itemset. Liu et al. [18] improved Qiu et al.'s work, which can protect the mining results well. However, a lot of dummy transactions are introduced during the query. Thus, it is not efficient for the large-scale dataset. Ma et al. also improved Qiu et al.'s work by designing a blocking algorithm [36]. However, in their scheme, the mining result is still leaked to the cloud. Wu et al. proposed a privacy-preserving association rules mining scheme by designing a lightweight asymmetric homomorphic encryption algorithm [37]. However, this cryptosystem is not semantically secure. The encryption is navies, the secret message, i.e., U, is just encrypted by multiplying an integer and then plus another random integer. In their scheme, the distribution of the ciphertext can be easily obtained. Therefore, with some other knowledge, the adversary can easily decrypt the ciphertext. Most recently, Pang et al. further improved the performance of Liu et al.'s work [19]. Most recently, Chen et al. designed a verifiable privacy-preserving association rule query scheme on the cloud. Nevertheless, in these, dummy transactions are required to be added, which makes these schemes inefficient.

IX. Conclusion

We proposed a privacy-preserving frequent itemsets and association rule mining and query scheme on outsourced data in a two-cloud model in this work. In our scheme, the CSP collected lots of encrypted data from multi-party and then can securely mine the frequent itemsets and association rules with the help of the Evaluator. After mining, the CSP also cooperates with Evaluator to provide a privacy-preserving frequent itemsets and association rule query service. Our scheme was built on a set of cryptographic building blocks. Specifically, the computation cost of the cloud in the query scheme is relatively small.

References

[1] S. Qiu, B. Wang, M. Li, J. Liu, and Y. Shi, "Toward practical privacy-preserving frequent itemset mining on encrypted cloud data," *IEEE Trans. Cloud Comput.*, vol. 8, no. 1, pp. 312–323, First Quarter 2020.

[2] S. Sarawagi and S. H. Nagaralu, "Data mining models as services on the internet," *ACM SIGKDD Explorations Newslett.*, vol. 2, no. 1, pp. 24–28, 2000.

[3] F. Giannotti, L. V. Lakshmanan, A. Monreale, D. Pedreschi, and H. Wang, "Privacy-preserving mining of association rules from outsourced transaction databases," *IEEE Syst. J.*, vol. 7, no. 3, pp. 385–395, Sep. 2013.

[4] X. Yi, F. Y. Rao, E. Bertino, and A. Bouguettaya, "Privacy-preserving association rule mining in cloud computing," in *Proc. ACM Symp. Inf. Comput. Commun. Secur.*, 2015, pp. 439–450.

[5] C.-H. Tai, P. S. Yu, and M.-S. Chen, "k-support anonymity based on pseudo taxonomy for outsourcing of frequent itemset mining," in *Proc. 16th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2010, pp. 473–482.

[6] N. Wang, X. Xiao, Y. Yang, Z. Zhang, Y. Gu, and G. Yu, "Privsuper: A superset-first approach to frequent itemset mining under differential privacy," in *Proc. IEEE Int. Conf. Data Eng.*, 2017, pp. 809–820.

[7] C. Gentry et al., "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput.*, 2009, pp. 169–178.

[8] S. Brin, R. Motwani, J. D. Ullman, and S. Tsur, "Dynamic itemset counting and implication rules for market basket data," *ACM SIGMOD Rec.*, vol. 26, no. 2, pp. 255–264, 1997.

[9] W. Lee et al., "Data mining approaches for intrusion detection," in *Proc. USENIX Secur. Symp.*, San Antonio, TX, 1998, pp. 79–93.

[10] C. Estan and G. Varghese, *New Directions in Traffic Measurement and Accounting*, vol. 32. New York, NY, USA: ACM, 2002.

[11] B. Mobasher, N. Jain, E.-H. Han, and J. Srivastava, "Web mining: Pattern discovery from world wide web transactions," Department of Computer Science, University of Minnesota, Tech. Rep. TR96–050, 1996.

[12] M. Kantarcioglu and C. Clifton, "Privacy-preserving distributed mining of association rules on horizontally partitioned data," *IEEE Trans. on Knowl. data Eng.*, vol. 16, no. 9, pp. 1026–1037, Sep. 2004.

[13] J. Vaidya and C. Clifton, "Privacy preserving association rule mining in vertically partitioned data," in *Proc. 8th ACM SIGKDD Int. Conf. Knowl. Discov. Data mining*, 2002, pp. 639–644.

[14] J. Lai, Y. Li, R. H. Deng, J. Weng, C. Guan, and Q. Yan, "Towards semantically secure outsourcing of association rule mining on categorical data," *Inf. Sci.*, vol. 267, no. 2, pp. 267–286, 2014.

[15] J. Liu, J. Li, S. Xu, and B. C. Fung, "Secure outsourced frequent pattern mining by fully homomorphic encryption," in *Proc. Int. Conf. Big Data Analytics Knowl. Discov.*, Springer, 2015, pp. 70–81.

[16] C. Huang and R. Lu, "EFPA: Efficient and flexible privacy-preserving mining of association rule in cloud," in *Proc. IEEE/CIC Int. Conf. Commun. China*, 2015, pp. 1–6.

[17] L. Li, R. Lu, K.-K. R. Choo, A. Datta, and J. Shao, "Privacy-preserving-outsourced association rule mining on vertically partitioned databases," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 8, pp. 1847–1861, Aug. 2016.

[18] L. Liu et al., "Privacy-preserving mining of association rule on outsourced cloud data from multiple parties," in *Proc. Australas. Conf. Inf. Secur. Privacy*, Springer, 2018, pp. 431–451.

[19] H. Pang and B. Wang, "Privacy-preserving association rule mining using homomorphic encryption in a multikey environment," *IEEE Syst. J.*, vol. 15, no. 2, pp. 3131–3141, Jun. 2021.

[20] Y. Chen, Q. Zhao, P. Duan, B. Zhang, Z. Hong, and B. Wang, "Verifiable privacy-preserving association rule mining using distributed decryption mechanism on the cloud," *Expert Syst. Appl.*, vol. 201, 2022, Art. no. 117086.

[21] R. Cramer and V. Shoup, "Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption," in *Proc. Int. Cryptol. Conf.*, 2002, pp. 45–64.

[22] E. Bresson, D. Catalano, and D. Pointcheval, "A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Springer, 2003, pp. 37–54.

[23] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," *ACM SIGMOD Rec.*, vol. 22, no. 2, 1993, pp. 207–216.

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.

[24] R. Agrawal et al., "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases*, 1994, pp. 487–499.
[25] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.
[26] D. Beaver, "Efficient multiparty protocols using circuit randomization," in *Proc. Annu. Int. Cryptol. Conf.*, Springer, 1991, pp. 420–432.
[27] T. Schneider and M. Zohner, "GMW vs. YAO? Efficient secure two-party computation with low depth circuits," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, Springer, 2013, pp. 275–292.
[28] M. S. Riazi, C. Weinert, O. Tkachenko, E. M. Songhori, T. Schneider, and F. Koushanfar, "Chameleon: A hybrid secure computation framework for machine learning applications," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2018, pp. 707–721.
[29] J. Kim, D. Koo, Y. Kim, H. Yoon, J. Shin, and S. Kim, "Efficient privacy-preserving matrix factorization for recommendation via fully homomorphic encryption," *ACM Trans. Privacy Secur.*, vol. 21, no. 4, 2018, Art. no. 17.
[30] T. Veugen, R. de Haan, R. Cramer, and F. Muller, "A framework for secure computations with two non-colluding servers and multiple clients, applied to recommendations," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 3, pp. 445–457, Mar. 2015.
[31] V. Nikolaenko, U. Weinsberg, S. Ioannidis, M. Joye, D. Boneh, and N. Taft, "Privacy-preserving ridge regression on hundreds of millions of records," in *Proc. IEEE Symp. Secur. Privacy*, 2013, pp. 334–348.
[32] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.
[33] W. Wu, U. Parampalli, J. Liu, and M. Xian, "Privacy preserving k-nearest neighbor classification over encrypted database in outsourced cloud environments," *World Wide Web*, vol. 22, no. 1, pp. 101–123, 2019.
[34] W. Wu, J. Liu, H. Wang, J. Hao, and M. Xian, "Secure and efficient outsourced k-means clustering using fully homomorphic encryption with ciphertext packing technique," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 10, pp. 3424–3437, Oct. 2021.
[35] S. Servan-Schreiber, S. Langowski, and S. Devadas, "Private approximate nearest neighbor search with sublinear communication," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 911–929.
[36] C. Ma, B. Wang, K. Jooste, Z. Zhang, and Y. Ping, "Practical privacy-preserving frequent itemset mining on supermarket transactions," *IEEE Syst. J.*, vol. 14, no. 2, pp. 1992–2002, Jun. 2020.
[37] J. Wu, N. Mu, X. Lei, J. Le, D. Zhang, and X. Liao, "SecEDMO: Enabling efficient data mining with strong privacy protection in cloud computing," *IEEE Trans. Cloud Comput.*, vol. 10, no. 1, pp. 691–705, First Quarter 2022.
[38] O. Goldreich, *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge, U.K.: Cambridge Univ. Press, 2009.
[39] D. Bogdanov, S. Laur, and J. Willemson, "Sharemind: A framework for fast privacy-preserving computations," in *Proc. 13th Eur. Symp. Res. Comput. Secur.*, Málaga, Spain, Springer, Oct. 6–8, 2008, pp. 192–206.
[40] E. Barker, W. Barker, W. Burr, W. Polk, and M. Smid, "NIST special publication 800–57," *NIST Special Publication*, vol. 800, no. 57, pp. 1–142, 2007.
[41] C. Dong and L. Chen, "A fast secure dot product protocol with application to privacy preserving association rule mining," in *Proc. Pacific-Asia Conf. Knowl. Discov. Data Mining*, Springer, 2014, pp. 606–617.
[42] C. Huang, R. Lu, and K.-K. R. Choo, "Secure and flexible cloud-assisted association rule mining over horizontally partitioned databases," *J. Comput. Syst. Sci.*, vol. 89, pp. 51–63, 2017.
[43] W. K. Wong, D. W. Cheung, E. Hung, B. Kao, and N. Mamoulis, "Security in outsourcing of association rule mining," in *Proc. Int. Conf. Very Large Data Bases*, 2007, pp. 111–122.
[44] I. Molloy, N. Li, and T. Li, "On the (In) security and (Im) practicality of outsourcing precise association rule mining," in *Proc. IEEE 9th Int. Conf. Data Mining*, 2009, pp. 872–877.
[45] S. Li, N. Mu, J. Le, and X. Liao, "Privacy preserving frequent itemset mining: Maximizing data utility based on database reconstruction," *Comput. Secur.*, vol. 84, pp. 17–34, 2019.
[46] C. Dong, L. Chen, and Z. Wen, "When private set intersection meets big data: An efficient and scalable protocol," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 789–800.



Lin Liu received the PhD degree in computer science from the National University of Defense Technology, Changsha, China, in 2020. Currently, he is an associate professor with the College of Computer, National University of Defense Technology, Changsha, China. His research interests include applied cryptography and privacy-preserving machine learning.



Jinshu Su received the BS degree in mathematics from Nankai University, Tianjin, China, in 1985, and the MS and PhD degrees in computer science from the National University of Defense Technology, Changsha, China, in 1988 and 2000, respectively. He is a professor with the School of Computer Science, National University of Defense Technology. He currently leads the distributed computing and high performance router laboratory and the computer networks and information security laboratory.



Ximeng Liu (Member, IEEE) received the BE degree from the Department of Electronic Engineering from Xidian University, Xi'an, China, in 2010 and the PhD degree from the Department of Telecommunication Engineering from Xidian University, Xi'an, China in 2015. He is now a full professor with the college of Mathematics and Computer Science, Fuzhou University, China. His research interests include applied cryptography and Big Data security. He has published more than 100 research articles including the *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Depenable and Secure Computing*, *IEEE Transactions on Computers* and IEEE INFOCOM. He awards "Minjiang Scholar Distinguished Professor", "Qishan Scholar in Fuzhou University" and ACM SIGSAC China Rising Star Award (2018). He is a member of ACM and CCF.



Rongmao Chen received the PhD degree in computer science from the University of Wollongong, Australia. He is now an associate professor with the College of Computer, National University of Defense Technology in China. His major research interests include applied cryptography and cybersecurity. He has published more than 50 research papers in refereed international conferences and journals, such as CRYPTO, ASIACRYPT, PKC, and CT-RSA. He has served as the program committee member on various international conferences such as ASIACRYPT, ESORICS, and AsiaCCS. He received the prestigious Young Elite Scientists Sponsorship by CAST (2017), Rising Star Award by ACM SIGSAC CHINA (2020), and Distinguished Young Scholars by National Natural Science Foundation of China (2021).

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.



Xinyi Huang received the PhD degree from the School of Computer Science and Software Engineering, University of Wollongong, Australia, in 2009. He is currently an associate professor with the Thrust of Artificial Intelligence, Information Hub, Hong Kong University of Science and Technology (Guangzhou), China. His research interests include cryptography and information security. He has published more than 160 research papers in refereed international conferences and journals, such as ACM CCS, Crypto, Asiacrypt, *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, and *IEEE Transactions on Information Security and Forensics*. His work has been cited more than 10000 times with Google Scholar. He is in the Editorial Board of *International Journal of Information Security* and *Science China Information Sciences*. He has served as the program/general chair or program committee member in more than 120 international conferences.



Guang Kou was born in 1983 and received the doctorate degree from Information Engineering University, in 2016. He is now an associate research fellow of the National Innovation Institute of Defense Technology, a master's tutor, and a master's tutor of Tianjin University. The main research areas are intelligent algorithm and intelligent security.



Shaojing Fu received the PhD degree in applied cryptography from the National University of Defense Technology in 2010 and has studied with University of Tokyo for one year during the PhD. He is currently an associate professor with the College of Computer, National University of Defense Technology. His research interests include cryptography theory and application, cloud storage security, Blockchain.

Authorized licensed use limited to: UNIV ESTADUAL PAULISTA JULIO DE MESQUITA FILHO. Downloaded on October 06,2026 at 22:30:59 UTC from IEEE Xplore. Restrictions apply.